

■ Design YouTube / Netflix

System Design Interview | Media Streaming Platform

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Upload video; Transcode to multiple qualities; Stream video; Search; Recommendations; Like/comment; Subscriptions

Non-Functional

2B users (YT); 500M hours watched/day; Upload: 500 hrs/min; Streaming latency <2s; 99.99% availability

Scale

Massive storage: 1hr 1080p $\approx$ 8GB; after compression $\sim$ 1GB; 500hrs/min $\times$ 60GB/hr = $\sim$ 30 TB/hr new content

■ High-Level Architecture

- Upload Service $\rightarrow$ Object Storage (raw S3) $\rightarrow$ Transcoding Pipeline $\rightarrow$ CDN origin buckets $\rightarrow$ CDN edge nodes
- Metadata Service stores video info, tags, channels; separate from media binary storage
- Streaming: Adaptive Bitrate (ABR) — HLS or DASH splits video into 2-10s segments; client picks quality based on bandwidth
- CDN edge nodes serve video segments; 95%+ cache hit rate; origin shield reduces S3 load

■ Video Processing Pipeline

- **Upload:** Client uploads raw video to S3 via pre-signed URL; event triggers SQS/Kafka message to processing queue
- **Transcode:** Worker pool (EC2/ECS) picks up job; FFmpeg transcodes to 360p/480p/720p/1080p/4K; parallelizable across workers
- **Segmentation:** Split each quality into 2-10s HLS/DASH segments; generate .m3u8 manifest files listing all segments
- **Storage:** Segments stored in S3 (origin); manifest uploaded; CDN pre-warms popular content; thumbnail generated
- **Status:** Progress tracked in DB; async notification to uploader; video available after processing completes

■ Adaptive Bitrate Streaming (ABR)

- MPEG-DASH / HLS: video split into small segments (2-10s); each segment available in multiple bitrates
- Client monitors download speed + buffer level $\rightarrow$ dynamically switches quality without interruption
- Startup optimization: begin with lower quality to fill buffer fast, then upgrade to higher quality
- Bandwidth estimation: EWMA of last N segment download times; switch up if throughput $>$ 1.5 $\times$ current bitrate

■ Data Model & Search

- videos: video_id, title, description, channel_id, status, duration, view_count, created_at (PostgreSQL)

- video_segments: stored as files in S3 with naming: {video_id}/{quality}/{segment_num}.ts
- Search: Elasticsearch index on title, description, tags; autocomplete with edge n-gram tokenizer
- View counts: async increment via Kafka → batch update DB every 60s (avoid write hotspot on popular videos)

■ CDN Strategy

Push CDN	Pull CDN
✓ Pre-load content to edge	✓ Cache on first request
✓ Better for predictable traffic	✓ Better for long-tail content
✓ Higher storage cost	✓ Origin hit on cold cache
✓ Netflix uses for new releases	✓ YouTube uses for most content

■ Recommendations Engine

- Collaborative filtering: users with similar watch history → recommend what they watched that you haven't
- Content-based: video embeddings (title, tags, transcript) → cosine similarity → similar videos
- Two-tower model: user tower (watch history, demographics) + item tower (video features) → dot product similarity
- Candidate generation (millions→hundreds) → Ranking model (hundreds→10s) → Re-ranking for diversity/freshness